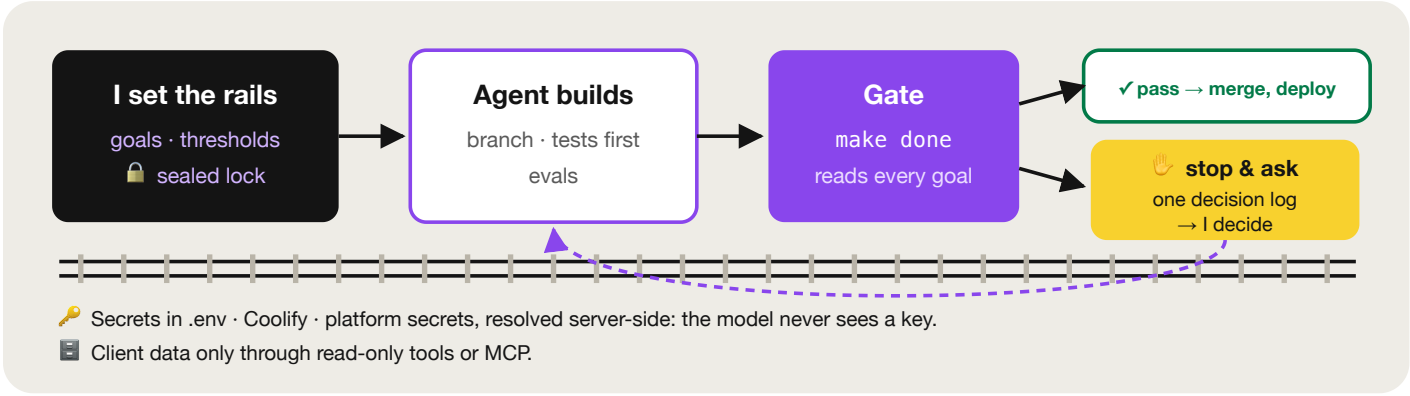
How I work with AI

Samuel Viciana · AI Forward Deployed Engineer · indigo.ai assignment, part 1

I don't guide the agent. **I put it on rails.** Claude Code writes most of the code; I decide what "done" means before it writes a line, and nothing it builds can move that line.



1 · My workflow

- **Rails first.** Each goal has a threshold, the command that measures it and the file it writes. A sha256 lock seals them and only I regenerate it: the agent reads goals, it cannot edit them.
- **One decision log.** If a goal misses after two measured attempts, or it needs money, a dependency, an outward action or a judgement, the agent stops, writes it down and asks me, with options and a recommendation.
- **Tests ≠ evals.** Ordinary backend code: tests first and a coverage gate. What a model consumes (tool descriptions, error messages, the prompt) is measured with a model: a hand-reviewed bank of tool calls, sentence→tool cases on a model, and end-to-end simulations read against the data.
- **Designing for a model is different.** A backend is judged by its tests; an interface an LLM consumes is judged by what a model does with it. So every response says what it could not match (missing terms, over budget, out of stock with alternatives) and every error says how to fix the call.
- **Secrets never reach a model.** The agent never reads keys; client data reaches the model only through read-only tools or MCP.

2 · When it went wrong

- **The agent took the shortcut.** I was building a system where a deep-learning model chose the next question to show a user. The client added a requirement: never repeat a question, whatever happens. I gave it to the coding agent. With no goals sealed by me and no single document logging each change, it met the rule by showing unseen questions without using the model at all, because that was simpler. That taught me to secure every step: I seal the goals, every change is logged in one place, and a miss stops the agent instead of letting it "fix" the goal.
- **Model-driven consumers fail silently.** When a model consumes what you build, nothing crashes: it just answers worse. A connector can drop the field you wrote; a judge model can fail correct answers because it cannot see the data. So I test on the real consumer, read the transcripts myself and instrument before guessing.
- **What I do differently now.** Every number comes with its command and its sample size, I read failures before accepting a verdict, and a prompt change is measured, not felt.

3 · In the room

I hold a position when I have evidence on the client's own data and I can show the trade-off: what happens, in a transcript or a measurement, not an opinion or a benchmark. I change it when the client brings a constraint I didn't know (cost, compliance, existing systems) or better evidence. Then I rerun the same test, write down the decision and why, and move on. What I never do is move the bar so a result passes.

Engineering is added value. SpaceX's value isn't metal and fuel; it's how hard it is to turn them into a rocket that comes back. Models are my raw material. My job is the hard part around them: using tools like **Claude Code** professionally, securely and continuously. Not guided: on rails.